Status	Finished
Started	Wednesday, 22 July 2026, 2:17 PM
Completed	Wednesday, 22 July 2026, 2:24 PM
Duration	6 mins 22 secs
Grade	Not yet graded

Question 1
Complete
Marked out of 5.00

In Experiment B, write-back was significantly faster than write-through. Describe exactly what data would be permanently lost if the cache-primary container crashed immediately after a write-back write (before the 3-second flush). How does write-through prevent this? When would you choose each strategy?

- Writeback writes Key/value only exists in the cache-primary that is marked as dirty and it is not sent to origin. So latest write lost forever.
- But write-through first write on the origin and only confirms success after origin says writes ok. So it's already durable on origin.
- Write-through – strong consistency needs – eg; bank balances, orders, critical records
- Writeback – usecases where performance(latency) more important than durability (session data, temp view counts)

Question **2**

Complete

Marked out of 5.00

Your LRU eviction removes the least recently used key. Describe a realistic scenario where LRU gives poor cache performance (hint: think about access patterns). What alternative eviction strategy would work better for that scenario?

- Sequential scan job that read big dataset (1000 keys one time each). It will remove all frequently used keys.
- LFU – Least Frequently Used – it keeps keys that are accessed lot over time

Question **3**

Complete

Marked out of 5.00

In Experiment E, replica1 missed writes that happened while it was stopped. When it restarted, it did not automatically catch up. Describe how you would add a catch-up mechanism — what information would the primary need to store, and what would the replica do on reconnection?

- Primary needs to have a write log(list of all writes in order with seq #)
- In Replica, it should remember last sequence number it applied.
- Once reconnect, replicas asks primary to give me everything after seq#
- So replica applies them in order to come to current state.

Question 4
Complete
Marked out of 5.00

Your cache is an AP system — it stays available but can return stale data during replication lag. Lab 03 showed that CP systems refuse requests to stay consistent. Describe a scenario where your cache returning stale data causes a real problem. How would you change the architecture to make it CP?

- Think of Usecase of inventory checking when user buy item, in primary cache has stock_quanity = 1, it replicates to replica1 and replica2. UserA hits replica1 and buys last item. Write goes to primary and primary updates stock_quantity=0 ->it starts replicating to replica1 and replica2. Before replication finishes(some lag there) CustomerB hits replica2 and sees stock_quanity=1 and successfully buys. But it already 0 so oversold happened.
- For every read, replica require to confirm with primary or with a quorum that its data is correct before answering. If it cant confirm, refuse/block the read instead of returing stale data.

Question 5
Complete
Not graded

Record your results from all experiments below. All tables are graded manually.

Table 1 — Experiments A and B: Latency

Measurement	Value
A — Cache miss latency (Round 1)	57.80ms
A — Cache hit latency (Round 2)	1.95ms
A — Hit rate after Round 2	68.8%
B — Average write-through latency	54.12ms
B — Average write-back latency	1.81ms
B — Speed difference (x times faster)	~29.9 times

Table 2 — Experiment C: LRU Eviction

Measurement	Value

Which key was evicted?	key1
Was it the LRU key? (Yes/No)	Yes
Getting evicted key after eviction — cache hit or miss?	Cache miss

Table 3 — Experiments D and E: Replication

Measurement	Value
D — Both replicas had all 5 keys immediately? (Yes/No)	Yes
D — Replica hit rate	100%
E — Reading from stopped replica — what happened?	get calls fails - connection error
E — After restart: replica had missed writes? (Yes/No)	No

Previous activity
Lab 06 Student Guide
Jump to...
Next activity
Lecture 14-BlockChain Principles